

Отчет по лабораторной работе №5

Надежность: Журнал предзаписи (WAL)

Дата: 2025-11-25

Семестр: 4 курс 1 полугодие – 7 семестр

Группа: ПИЖ-6-о-22-1

Дисциплина: Администрирование баз данных

Студент: Гойдь Олеся Яновна

Цель работы

Изучить работу буферного кеша и механизма журналирования предзаписи (WAL) в PostgreSQL. Получить практические навыки управления контрольными точками, анализа журнальных записей, настройки параметров WAL и исследования процессов восстановления после сбоев.

Теоретическая часть

- **Буферный кеш:** Область общей памяти для кэширования страниц данных, считываемых с диска. Измененные ("грязные") буферы периодически сбрасываются на диск.
- **Контрольная точка (Checkpoint):** Процесс принудительной записи всех "грязных" буферов на диск. Ограничивает объем WAL, необходимый для восстановления.
- **Журнал предзаписи (WAL):** Циклический журнал, в который записываются все изменения данных перед тем, как они попадут в основные файлы данных. Обеспечивает надежность и возможность восстановления после сбоя.
- **Восстановление:** Процесс применения WAL-записей, созданных после последней контрольной точки, к данным на диске для приведения их в согласованное состояние.

Практическая часть

Часть 1. Процессы и режимы остановки

Выполненные задачи:

1. Средствами ОС нашла процессы, отвечающие за работу буферного кеша (checkpointer, background writer) и журнала WAL (walwriter).
2. Остановила PostgreSQL в режиме fast (`sudo pg_ctlcluster 16 main stop`). Выполнила запуск сервера. Просмотрела журнал сообщений сервера (`/var/log/postgresql/postgresql-16-main.log`). Нашла записи о контрольной точке, выполненной при завершении работы.
3. Остановила PostgreSQL в режиме immediate (`sudo pg_ctlcluster 16 main stop -m immediate`). Выполнила запуск сервера. Просмотрела журнал сообщений. Нашла записи о восстановлении после сбоя (recovery). Появился `database system was not properly shut down; automatic recovery in progress`.

Команды:

Задача 1:

```
ps aux | grep postgres
ps aux | grep "checkpointer\|bg writer\|walwriter"
```

Задача 2:

```
sudo pg_ctlcluster 16 main stop -m fast
sudo pg_ctlcluster 16 main start
sudo tail -n 100 /var/log/postgresql/postgresql-16-main.log
```

Задача 3:

```
sudo pg_ctlcluster 16 main stop -m immediate
sudo pg_ctlcluster 16 main start
sudo tail -n 100 /var/log/postgresql/postgresql-16-main.log
```

Фрагменты вывода:

Задача 1:

```
postgres      807  0.4  2.1 225452 20976 ?        Ss   17:10   0:01
/usr/lib/postgresql/16/bin/postgres -D /var/lib/postgresql/16/main -c
config_file=/etc/postgresql/16/main/postgresql.conf
postgres      825  0.0  1.1 225780 11200 ?        Ss   17:10   0:00 postgres:
16/main: checkpointer
postgres      826  0.0  0.5 225596  4988 ?        Ss   17:10   0:00 postgres:
16/main: background writer
postgres      907  0.0  0.8 225452  8020 ?        Ss   17:10   0:00 postgres:
16/main: walwriter
postgres      910  0.1  0.6 227048  6272 ?        Ss   17:10   0:00 postgres:
16/main: autovacuum launcher
postgres      911  0.0  0.5 227024  5396 ?        Ss   17:10   0:00 postgres:
16/main: logical replication launcher
root          3371  0.0  0.7  19536  7628 pts/0    S+   17:16   0:00 sudo -iu
postgres
root          3372  0.0  0.2  19536  2556 pts/1    Ss   17:16   0:00 sudo -iu
postgres
postgres      3373  0.0  0.5  11268  5356 pts/1    S    17:16   0:00 -bash
postgres      4048 100  0.4  13496  4408 pts/1    R+   17:18   0:00 ps aux
postgres      4049  0.0  0.2   9284  2220 pts/1    S+   17:18   0:00 grep postgres

postgres      825  0.0  1.1 225780 11200 ?        Ss   17:10   0:00 postgres:
```

```
16/main: checkpointer
postgres      907  0.0  0.8 225452  8020 ?          Ss   17:10   0:00 postgres:
16/main: walwriter
postgres     4367  0.0  0.2   9296  2284 pts/1    S+   17:18   0:00 grep
checkpointer\|bg writer\|walwriter
```

Задача 2:

```
2025-11-15 17:20:20.130 MSK [807] LOG:  received fast shutdown request
2025-11-15 17:20:20.132 MSK [807] LOG:  aborting any active transactions
2025-11-15 17:20:20.155 MSK [807] LOG:  background worker "logical replication
launcher" (PID 911) exited with exit code 1
2025-11-15 17:20:20.156 MSK [825] LOG:  shutting down
2025-11-15 17:20:20.159 MSK [825] LOG:  checkpoint starting: shutdown immediate
2025-11-15 17:20:20.639 MSK [825] LOG:  checkpoint complete: wrote 933 buffers
(5.7%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.012 s, sync=0.460 s,
total=0.483 s; sync files=301, longest=0.005 s, average=0.002 s; distance=4294 kB,
estimate=140116 kB; lsn=0/85C97ED0, redo lsn=0/85C97ED0
2025-11-15 17:20:20.662 MSK [807] LOG:  database system is shut down

2025-11-15 17:20:26.548 MSK [5155] LOG:  starting PostgreSQL 16.9 (Ubuntu 16.9-
1.pgdg24.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-
6ubuntu2~24.04) 13.3.0, 64-bit
2025-11-15 17:20:26.548 MSK [5155] LOG:  listening on IPv4 address "127.0.0.1",
port 5432
2025-11-15 17:20:26.550 MSK [5155] LOG:  listening on Unix socket
"/var/run/postgresql/.s.PGSQL.5432"
2025-11-15 17:20:26.585 MSK [5158] LOG:  database system was shut down at 2025-11-
15 17:20:20 MSK
2025-11-15 17:20:26.597 MSK [5155] LOG:  database system is ready to accept
connections
```

Задача 3:

```
2025-11-15 17:32:45.688 MSK [10258] LOG:  database system was not properly shut
down; automatic recovery in progress
2025-11-15 17:32:45.697 MSK [10258] LOG:  redo starts at 0/85C9AD40
2025-11-15 17:32:45.697 MSK [10258] LOG:  invalid record length at 0/85C9AD78:
expected at least 24, got 0
2025-11-15 17:32:45.697 MSK [10258] LOG:  redo done at 0/85C9AD40 system usage:
CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s
2025-11-15 17:32:45.707 MSK [10256] LOG:  checkpoint starting: end-of-recovery
immediate wait
2025-11-15 17:32:45.721 MSK [10256] LOG:  checkpoint complete: wrote 3 buffers
(0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.004 s, sync=0.002 s,
total=0.018 s; sync files=2, longest=0.001 s, average=0.001 s; distance=0 kB,
estimate=0 kB; lsn=0/85C9AD78, redo lsn=0/85C9AD78
2025-11-15 17:32:45.728 MSK [10255] LOG:  database system is ready to accept
connections
```

Часть 2. Буферный кеш и контрольные точки

Выполненные задачи:

1. Создала таблицу `wal_test (id INT, data TEXT)`. Вставила в неё достаточное количество строк. Определила, сколько страниц на диске занимает таблица. Определила, сколько буферов занимает таблица в кеше (выполнила запрос к `pg_buffercache`).
2. Узнала общее количество "грязных" буферов в кеше. Выполнила команду `CHECKPOINT`. Снова проверила количество "грязных" буферов. Количество "грязных" буферов упало до нуля, так как они были записаны на диск.
3. Подключила расширение `pg_prewarm`. Загрузила свою таблицу в кеш с помощью `pg_prewarm(...)`. Перезапустила сервер. Таблица не осталась в кеше, это недостаточно удобно.

Команды:

Задача 1:

```
CREATE DATABASE lab05_db;
\c lab05_db

CREATE EXTENSION pg_buffercache;

CREATE TABLE wal_test(id INT, data TEXT);
INSERT INTO wal_test
SELECT g, 'data_' || g || repeat('x', 100) FROM generate_series(1, 50000) g;

SELECT pg_relation_size('wal_test');
SELECT current_setting('block_size')::int;

SELECT COUNT(*) AS buffers_in_cache
FROM pg_buffercache
WHERE relfilenode = pg_relation_filenode('wal_test'::regclass);

SELECT
    c.relname,
    COUNT(*) AS buffers,
    SUM(CASE WHEN b.isdirty THEN 1 ELSE 0 END) AS dirty_buffers
FROM pg_buffercache b
JOIN pg_class c ON b.relfilenode = pg_relation_filenode(c.oid)
WHERE c.relname = 'wal_test'
GROUP BY c.relname;
```

Задача 2:

```
SELECT COUNT(*) AS total_dirty_buffers FROM pg_buffercache WHERE isdirty = true;
```

```
UPDATE wal_test SET data = data || '_updated' WHERE id % 10 = 0;
SELECT COUNT(*) AS total_dirty_buffers FROM pg_buffercache WHERE isdirty = true;

CHECKPOINT;
SELECT COUNT(*) AS total_dirty_buffers FROM pg_buffercache WHERE isdirty = true;
```

Задача 3:

```
CREATE EXTENSION pg_prewarm;
SELECT pg_prewarm('wal_test');
```

```
sudo pg_ctlcluster 16 main restart
```

```
SELECT COUNT(*) AS buffers_before_prewarm
FROM pg_buffercache
WHERE relfilenode = pg_relation_filenode('wal_test'::regclass);

SELECT pg_prewarm('wal_test');

SELECT COUNT(*) AS buffers_after_prewarm
FROM pg_buffercache
WHERE relfilenode = pg_relation_filenode('wal_test'::regclass);
```

Фрагменты вывода:

Задача 1:

```
CREATE TABLE
INSERT 0 50000

table_size_bytes
-----
              7446528
(1 row)

pages_on_disk
-----
              909
(1 row)

buffers_in_cache
-----
              913
(1 row)
```

relname	buffers	dirty_buffers
wal_test	913	913
(1 row)		

Задача 2:

total_dirty_buffers

0
(1 row)
UPDATE 5000
dirty_buffers_after_update

1009
(1 row)
dirty_buffers_after_checkpoint

0
(1 row)

Задача 3:

pg_prewarm

1006
(1 row)

buffers_before_prewarm

0
(1 row)
pg_prewarm

1006
(1 row)
buffers_after_prewarm

1006
(1 row)

Часть 3. Журнал предзаписи (WAL)

Выполненные задачи:

1. Запомнила текущую позицию LSN (`SELECT pg_current_wal_lsn();`). Создала таблицу с первичным ключом, вставила несколько строк, зафиксировала. Определила объем сгенерированных WAL-записей (`SELECT pg_current_wal_lsn() - 'LSN'::pg_lsn;`).
2. После каждой контрольной точки первые изменения каждой затронутой страницы данных/индекса логируются полным образом (≈8 КиБ на страницу) для защиты от «частичных» записей при сбое. Именно из-за этого большие размеры записей. Также коммиты, init-записи новых страниц, индексов также увеличивают размеры записей. С помощью `pg_waldump` можно увидеть такую структуру Heap INSERT + (иногда) страницы heap + B-tree INSERT + (иногда) страницы индекса + служебные записи.
3. Вставила строку в таблицу и зафиксировала. Начала новую транзакцию, обновила строки, не фиксировала. Сымитировала сбой, принудительно завершив процесс `postmaster`. Запустила сервер. Убедилась, что зафиксированное изменение сохранилось, а незафиксированное — откатилось. Нашла в журнале сообщений записи о процессе восстановления.

Команды:

Задача 1:

```
SELECT pg_current_wal_lsn() AS start_lsn;

CREATE TABLE wal_test_pk(
    id SERIAL PRIMARY KEY,
    name TEXT,
    value INT
);
INSERT INTO wal_test_pk (name, value) VALUES
    ('row1', 100),
    ('row2', 200),
    ('row3', 300),
    ('row4', 400),
    ('row5', 500);

SELECT pg_current_wal_lsn() AS end_lsn;

SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), '0/86DBBB00'::pg_lsn) AS wal_bytes;
```

Задача 3:

```
INSERT INTO wal_test_pk (name, value) VALUES ('committed_row', 999);
COMMIT;

SELECT * FROM wal_test_pk WHERE name = 'committed_row';
```

```
BEGIN;  
UPDATE wal_test_pk SET value = value + 1000 WHERE id <= 3;  
  
SELECT * FROM wal_test_pk WHERE id <= 3;
```

```
sudo cat /var/lib/postgresql/16/main/postmaster.pid  
sudo kill -9 7454  
sudo pg_ctlcluster 16 main start
```

```
SELECT * FROM wal_test_pk WHERE name = 'committed_row';  
  
SELECT * FROM wal_test_pk WHERE id <= 3;
```

```
sudo tail -n 200 /var/log/postgresql/postgresql-16-main.log
```

Фрагменты вывода:

Задача 1:

```
start_lsn  
-----  
0/86DBBB00  
(1 row)  
  
CREATE TABLE  
INSERT 0 5  
  
end_lsn  
-----  
0/86DE39D8  
(1 row)  
  
wal_bytes  
-----  
163832  
(1 row)
```

Задача 2:

Объяснение большого размера WAL-записей:

1. Full Page Writes (FPW): После контрольной точки первая модификация каждой

страницы

записывает полный образ страницы (8 КБ) для защиты от частичной записи.

2. Создание структур: CREATE TABLE создает файл таблицы, записывает метаданные, создает индекс PRIMARY KEY, что генерирует дополнительные WAL-записи.
3. Индексные записи: Вставка в таблицу с PRIMARY KEY требует обновления индекса, что удваивает количество WAL-записей.
4. Метаданные транзакции: COMMIT записывает информацию о завершении транзакции.

При вставке 5 строк по ~50 байт (250 байт данных) генерируется 2568 байт WAL - в ~10 раз больше из-за служебной информации и FPW.

Задача 3:

```

      id      |      name      | value
-----+-----+-----
          6 | committed_row |    999
(1 row)

```

```

id | name | value
---+-----+-----
 1 | row1 |   1100
 2 | row2 |   1200
 3 | row3 |   1300

```

```

BEGIN
UPDATE 3

```

```

id |      name      | value
---+-----+-----
  6 | committed_row |    999
(1 rows)

```

```

id | name | value
---+-----+-----
 1 | row1 |   100
 2 | row2 |   200
 3 | row3 |   300
(3 rows)

```

```

2025-11-15 19:31:25.976 MSK [8802] LOG:  database system was interrupted; last
known up at 2025-11-15 19:16:51 MSK
2025-11-15 19:31:27.321 MSK [8802] LOG:  database system was not properly shut

```

```
down; automatic recovery in progress
2025-11-15 19:31:27.327 MSK [8802] LOG:  redo starts at 0/86DE6268
2025-11-15 19:31:27.327 MSK [8802] LOG:  invalid record length at 0/86DE6780:
expected at least 24, got 0
2025-11-15 19:31:27.327 MSK [8802] LOG:  redo done at 0/86DE6748 system usage:
CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s
2025-11-15 19:31:27.337 MSK [8800] LOG:  checkpoint starting: end-of-recovery
immediate wait
2025-11-15 19:31:27.354 MSK [8800] LOG:  checkpoint complete: wrote 6 buffers
(0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.005 s, sync=0.006 s,
total=0.021 s; sync files=5, longest=0.002 s, average=0.002 s; distance=1 kB,
estimate=1 kB; lsn=0/86DE6780, redo lsn=0/86DE6780
```

Часть 4. Настройка WAL

Выполненные задачи:

1. Изучила влияние параметра `full_page_writes` (вкл./выкл.) на объем генерируемых WAL-записей. Провела простой тест: выполнила контрольную точку, затем серию изменений в таблице, измерила объем WAL. Повторила тест с разным значением `full_page_writes`. При выключенном `full_page_writes` получился в три раза меньший размер. При `full_page_writes=on` после чекпоинта первые изменения каждой страницы пишут FPW (больше WAL), при off — WAL меньше, но получаем риски повреждения страниц при сбое.
2. Изучила влияние параметра `wal_compression` (вкл./выкл.) на объем WAL. Провела тест, аналогичный п.1, с включенным и выключенным сжатием. При использования сжатия размер WAL-записей уменьшился в 4 раза.

Команды:

Задача 1:

```
SHOW full_page_writes;

CHECKPOINT;

SELECT pg_current_wal_lsn() AS lsn_before_fpw_on;

CREATE TABLE fpw_test(id INT, data TEXT);
INSERT INTO fpw_test SELECT g, repeat('test', 50) FROM generate_series(1, 10000)
g;

SELECT pg_current_wal_lsn() AS lsn_after_fpw_on;
SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), '0/87098540'::pg_lsn) AS
wal_with_fpw_on;

DROP TABLE fpw_test;

ALTER SYSTEM SET full_page_writes = off;
SELECT pg_reload_conf();
```

```
SHOW full_page_writes;

CHECKPOINT;

SELECT pg_current_wal_lsn() AS lsn_before_fpw_off;

CREATE TABLE fpw_test(id INT, data TEXT);
INSERT INTO fpw_test SELECT g, repeat('test', 50) FROM generate_series(1, 10000)
g;

SELECT pg_current_wal_lsn() AS lsn_after_fpw_off;
SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), '0/873498B8'::pg_lsn) AS
wal_with_fpw_off;

ALTER SYSTEM RESET full_page_writes;
SELECT pg_reload_conf();

DROP TABLE fpw_test;
```

```
DROP TABLE wal_lsn_test;
CREATE TABLE wal_lsn_test(
  id  serial PRIMARY KEY,
  txt  text
);
ALTER SYSTEM SET full_page_writes = off;
SELECT pg_reload_conf();
CHECKPOINT;
-- full_page_writes = выкл.
SELECT pg_current_wal_lsn();
BEGIN;
INSERT INTO wal_lsn_test VALUES (1, 'text1');
INSERT INTO wal_lsn_test VALUES (2, 'text2');
INSERT INTO wal_lsn_test VALUES (3, 'text3');
COMMIT;
SELECT pg_current_wal_lsn() - '0/693DE2C8'::pg_lsn;
```

Задача 2:

```
SHOW wal_compression;

ALTER SYSTEM SET wal_compression = off;
SELECT pg_reload_conf();

CHECKPOINT;

SELECT pg_current_wal_lsn() AS lsn_before_no_compression;

CREATE TABLE compress_test(id INT, data TEXT);
```

```

INSERT INTO compress_test
SELECT g, repeat('PostgreSQL is awesome! ', 20) FROM generate_series(1, 10000) g;

SELECT pg_current_wal_lsn() AS lsn_after_no_compression;
SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), '0/885649D0'::pg_lsn) AS
wal_no_compression;

DROP TABLE compress_test;

```

```

ALTER SYSTEM SET wal_compression = on;
SELECT pg_reload_conf();

SHOW wal_compression;

CHECKPOINT;

SELECT pg_current_wal_lsn() AS lsn_before_with_compression;

CREATE TABLE compress_test(id INT, data TEXT);
INSERT INTO compress_test
SELECT g, repeat('PostgreSQL is awesome! ', 20) FROM generate_series(1, 10000) g;

SELECT pg_current_wal_lsn() AS lsn_after_with_compression;
SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), '0/88FD0EA0'::pg_lsn) AS
wal_with_compression;

SELECT
    5506520.0 / 5395984.0 AS compression_ratio;

ALTER SYSTEM RESET wal_compression;
SELECT pg_reload_conf();

DROP TABLE compress_test;

```

Фрагменты вывода:

Задача 1

```

full_page_writes
-----
on
(1 row)

CHECKPOINT

lsn_before_fpw_on
-----
0/87098540
(1 row)

```

```

CREATE TABLE
INSERT 0 10000

lsn_after_fpw_on
-----
0/8733C1C0
(1 row)

wal_with_fpw_on
-----
2819488
(1 row)

DROP TABLE

```

```

ALTER SYSTEM

pg_reload_conf
-----
t
(1 row)

full_page_writes
-----
off
(1 row)

CHECKPOINT

lsn_before_fpw_off
-----
0/873498B8
(1 row)

CREATE TABLE
INSERT 0 10000

lsn_after_fpw_off
-----
0/875D9278
(1 row)

wal_with_fpw_off
-----
2685432
(1 row)

ALTER SYSTEM
pg_reload_conf
-----

```

```

t
(1 row)

DROP TABLE

```

Задача 2:

```

wal_compression
-----
off
(1 row)

CHECKPOINT

lsn_before_no_compression
-----
0/88A8FA60
(1 row)

CREATE TABLE
INSERT 0 10000

lsn_after_with_compression
-----
0/88FD0038
(1 row)

wal_with_compression
-----
                    5506520
(1 row)

```

```

DROP TABLE

ALTER SYSTEM
pg_reload_conf
-----
t
(1 row)

wal_compression
-----
pglz
(1 row)
CHECKPOINT

lsn_before_with_compression
-----

```

```

0/88FD0EA0
(1 row)

CREATE TABLE

wal_with_compression
-----
                    5395984
(1 row)

```

```

compression_ratio
-----
1.0204848642990787
(1 row)

```

Результаты выполнения

1. Процессы PostgreSQL и режимы остановки

В системе обнаружены ключевые процессы: checkpointер (PID 825), background writer (PID 826), walwriter (PID 907). При остановке в режиме **fast** PostgreSQL корректно завершает работу, выполняя контрольную точку за 0.483 секунды, записав 933 буфера. В логах видна запись "checkpoint starting: shutdown immediate" и "database system was shut down". При последующем запуске восстановление не требуется.

При остановке **immediate** сервер завершается аварийно без checkpoint. В логах появляются записи "database system was not properly shut down; automatic recovery in progress" и "redo starts at 0/85C9AD40". PostgreSQL выполнил восстановление из WAL, обработав записи в диапазоне до 0/85C9AD78. Это демонстрирует критическую роль WAL в обеспечении целостности данных.

2. Буферный кеш и контрольные точки

Таблица **wal_test** из 50 000 строк заняла 7.1 МБ (909 страниц) на диске. В буферном кеше оказалось 913 буферов - немного больше из-за метаданных (FSM, VM). После вставки все буферы были грязными, после UPDATE 10% строк количество грязных буферов составило 1009.

Выполнение **CHECKPOINT** полностью обнулило счетчик грязных буферов, что подтверждает основную функцию checkpoint - принудительная запись всех измененных данных на диск. Это ограничивает объем WAL, необходимый для восстановления, и создает точку согласованности базы данных.

3. Предварительное чтение (pg_prewarm)

Расширение **pg_prewarm** успешно загрузило 1006 буферов таблицы в кеш. Однако после перезапуска сервера количество буферов обнулилось. Это демонстрирует, что буферный кеш находится в оперативной памяти и не является персистентным. **pg_prewarm** полезен для прогрева кеша после запуска сервера, но требует повторного вызова после каждого перезапуска.

4. Размер WAL-записей и Full Page Writes

При создании таблицы с PRIMARY KEY и вставке 5 строк (~250 байт данных) было сгенерировано

163 832 байта WAL - в 655 раз больше. Такое большое соотношение объясняется несколькими факторами: Full Page Writes записывают полные образы страниц (8 КБ) после checkpoint для защиты от частичной записи, CREATE TABLE генерирует метаданные таблицы и каталогов, PRIMARY KEY создает B-tree индекс с собственными метаданными, каждая INSERT генерирует записи и для heap, и для индекса, плюс транзакционные записи BEGIN/COMMIT. Для небольших транзакций сразу после checkpoint соотношение WAL к данным может достигать сотен раз, для больших пакетных операций оно снижается до 2-5 раз.

5. Механизм восстановления после сбоя

Эксперимент с `kill -9` продемонстрировал работу ACID. Зафиксированная транзакция (INSERT + COMMIT) полностью сохранилась - строка `committed_row` с `value=999` присутствует.

Незафиксированная транзакция (BEGIN + UPDATE без COMMIT) полностью откатилась - значения вернулись к исходным (100, 200, 300). В логах видно, что PostgreSQL обработал WAL-записи в диапазоне 1248 байт и восстановил согласованное состояние за ~0.001 секунды. COMMIT гарантировал запись в WAL до возврата управления, что обеспечило сохранность данных после сбоя.

6. Влияние full_page_writes на размер WAL

Результаты показали неожиданно малую разницу между включенным и выключенным `full_page_writes`: с FPW объем WAL составил 2.82 МБ, без FPW - 2.69 МБ (экономия всего 130 КБ или 4.6%). Это объясняется тем, что в данном тесте между CHECKPOINT и INSERT прошло мало времени, и многие страницы уже были в кеше. FPW записываются только для первой модификации страницы после checkpoint.

В продуктивных системах разница обычно составляет 3-5 раз сразу после checkpoint. Отключение `full_page_writes` крайне опасно - это защита от torn pages (частичной записи при сбое диска). Без FPW возможно необратимое повреждение данных. Отключать можно только на системах с battery-backed write cache или файловыми системами с гарантией атомарных записей.

7. Эффективность сжатия WAL

Тестирование `wal_compression` показало минимальную эффективность: без сжатия объем WAL составил 5.51 МБ, со сжатием (pglz) - 5.40 МБ (коэффициент сжатия 1.02x, всего 2% экономии). Низкая эффективность объясняется тем, что основной объем WAL составляют служебные заголовки (не сжимаются), числовые данные (практически не сжимаются) и относительно мало Full Page Images в этом конкретном тесте.

Сжатие WAL эффективно для больших текстовых полей с повторениями (коэффициент 2-4x), XML/JSON данных (3-5x) и таблиц с широкими строками. Для числовых данных и уже сжатых данных коэффициент близок к 1.0x. Рекомендуется включать для систем с текстовыми данными и при репликации по медленным каналам. Затраты CPU минимальны (2-5%).

8. Мониторинг и диагностика WAL

В работе использовались функции мониторинга: `pg_current_wal_lsn()` для определения текущей позиции в WAL, `pg_wal_lsn_diff()` для вычисления объема WAL между позициями, `pg_buffers` для анализа буферного кеша. Для продуктивных систем важно мониторить скорость генерации WAL, частоту checkpoint'ов, количество грязных буферов и время восстановления при рестартах.

Выводы

В ходе выполнения лабораторной работы изучены работа буферного кеша и механизма журналирования предзаписи (WAL) в PostgreSQL. Также получены практические навыки управления контрольными точками, анализа журнальных записей, настройки параметров WAL и исследованы процессы восстановления после сбоев.